

TUGAS SCALABLE SYSTEMS DESIGN

Dosen pengampu: Runal Rezkiawan, S.Kom., M.T



OLEH:

NAMA	:	ALHIZRA
NIM	:	105841101523
KELAS	:	6.A

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH MAKASSAR
2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Allah SWT karena atas rahmat dan karunia-Nya, makalah yang berjudul “*Teorema CAP dalam Sistem Terdistribusi*” ini dapat diselesaikan dengan baik. Makalah ini disusun untuk memenuhi tugas mata kuliah *Scalable Systems Design* yang diampu oleh Runal Rezkiawan.

Penulis menyadari bahwa perkembangan teknologi informasi saat ini menuntut sistem komputer untuk mampu bekerja secara cepat, stabil, dan dapat melayani banyak pengguna secara bersamaan. Oleh karena itu, pemahaman mengenai sistem terdistribusi serta Teorema CAP menjadi sangat penting bagi mahasiswa informatika maupun praktisi teknologi dalam merancang sistem yang andal dan efisien.

Dalam makalah ini, penulis membahas mengenai pengertian Teorema CAP, komponen utama yang terdiri dari *Consistency*, *Availability*, dan *Partition Tolerance*, serta penerapannya dalam pemilihan teknologi database modern. Penulis berharap pembahasan ini dapat menambah wawasan pembaca mengenai konsep dasar scalable system dan tantangan dalam pengelolaan data terdistribusi.

Penulis menyadari bahwa makalah ini masih memiliki banyak kekurangan, baik dari segi penyusunan maupun isi materi. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan di masa yang akan datang.

Akhir kata, penulis berharap makalah ini dapat memberikan manfaat bagi pembaca dan menjadi referensi tambahan dalam mempelajari sistem terdistribusi.

Makassar, 15 Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI	ii
BAB I	1
PENDAHULUAN.....	1
A. Latar Belakang.....	1
B. Tujuan	2
BAB II.....	3
PEMBAHASAN	3
A. Definisi Teorema CAP	3
B. Komponen Utama Teorema CAP.....	3
C. Trade-off dan Strategi Implementasi	4
D. Relevansi CAP dalam Pemilihan Database (SQL vs NoSQL).....	7
BAB III.....	8
PENUTUP	8
A. Kesimpulan	8
B. Saran	8
DAFTAR PUSTAKA	9

BAB I

PENDAHULUAN

A. Latar Belakang

Di era transformasi digital saat ini, kebutuhan akan ketersediaan data secara real-time dan skalabilitas sistem menjadi prioritas utama bagi pengembang perangkat lunak. Seiring dengan pertumbuhan pengguna yang eksponensial, infrastruktur berbasis server tunggal (centralized system) mulai menemui titik jenuh, baik dari sisi performa maupun keandalan. Hal ini memicu adopsi luas terhadap sistem terdistribusi, di mana beban kerja dibagi ke berbagai node independen yang saling terhubung untuk memberikan kesan satu kesatuan sistem yang utuh bagi pengguna akhir.

Namun, mengelola data pada banyak lokasi geografis yang berbeda membawa tantangan teknis yang kompleks. Masalah utama muncul ketika jaringan yang menghubungkan node-node tersebut mengalami gangguan atau latensi tinggi. Dalam kondisi seperti ini, arsitek sistem harus mengambil keputusan sulit mengenai bagaimana data dikelola agar tetap konsisten tanpa mengorbankan aksesibilitas layanan. Fenomena dilematis ini dirumuskan dalam Teorema CAP, sebuah fondasi teoretis yang menjelaskan batas-batas intrinsik dalam penyimpanan data terdistribusi. Pemahaman yang keliru terhadap prinsip ini sering kali berakibat pada kegagalan sistem dalam menjaga integritas data atau menyebabkan downtime yang merugikan secara bisnis.

A. Rumusan Masalah

Berdasarkan latar belakang tersebut, maka rumusan masalah dalam makalah ini adalah:

- 1) Apa definisi mendasar dari Teorema CAP dan bagaimana sejarah singkat kemunculannya dalam arsitektur sistem terdistribusi?
- 2) Bagaimana karakteristik masing-masing komponen utama – Consistency, Availability, dan Partition Tolerance – didefinisikan secara teknis?
- 3) Mengapa sebuah sistem terdistribusi mustahil memenuhi ketiga elemen CAP secara bersamaan?
- 4) Bagaimana aplikasi praktis dari pemilihan kombinasi CAP (CP vs AP) terhadap pemilihan teknologi database dan keandalan aplikasi di dunia nyata?

B. Tujuan

Adapun tujuan dari penulisan makalah ini adalah sebagai berikut:

- 1) Memberikan penjelasan komprehensif mengenai batasan-batasan logis dalam sistem data terdistribusi melalui kerangka kerja Teorema CAP.
- 2) Menganalisis trade-of (pertukaran kepentingan) yang terjadi ketika perancang sistem harus memilih antara konsistensi data dan ketersediaan layanan.
- 3) Membantu pembaca, khususnya mahasiswa informatika dan praktisi teknologi, dalam menentukan strategi desain arsitektur yang paling tepat sesuai dengan kebutuhan fungsional aplikasi yang dibangun.
- 4) Memetakan berbagai teknologi database modern (SQL dan NoSQL) ke dalam kategori kombinasi CAP untuk mempermudah pengambilan keputusan teknis.

BAB II

PEMBAHASAN

A. Definisi Teorema CAP

Teorema CAP, yang juga dikenal sebagai Teorema Brewer, pertama kali diperkenalkan oleh ilmuwan komputer Eric Brewer pada tahun 2000. Teorema ini menjadi prinsip fundamental dalam desain sistem terdistribusi yang menyatakan bahwa sebuah sistem penyimpanan data bersama (shared-data system) secara teoretis mustahil untuk memenuhi tiga jaminan kualitas secara bersamaan, yaitu: Consistency, Availability, dan Partition Tolerance.

Dalam arsitektur sistem skala besar, teorema ini berfungsi sebagai kompas bagi para arsitek perangkat lunak untuk memahami batasan logis jaringan. Keberadaan teorema ini memaksa perancang sistem untuk menentukan prioritas berdasarkan kasus penggunaan (use case) spesifik aplikasi, karena saat terjadi gangguan jaringan, sistem harus memilih antara menjaga keakuratan data atau menjaga kelangsungan layanan.

B. Komponen Utama Teorema CAP

1. Consistency (Konsistensi)

Konsisten dalam konteks CAP merujuk pada Atomic Consistency atau Linearizability. Jaminan ini memastikan bahwa setiap operasi pembacaan (read) yang dilakukan setelah operasi penulisan (write) berhasil, akan selalu mengembalikn nilai terbaru atau pesan kesalahan (error).

Mekanisme: Dalam sistem yang konsisten, semua node atau server dalam kluster akan melihat urutan pembaruan data yang sama. Jika pengguna memperbarui data pada Node A, maka sistem harus memastikan bahwa Node B, C, dan seterusnya telah menerima pembaruan tersebut sebelum memberikan akses baca kepada pengguna lain.

Implikasi: Sistem yang sangat konsisten sangat krusial untuk transaksi finansial, di mana perbedaan saldo antar node dapat berakibat fatal.

2. Availability (Ketersediaan)

Ketersediaan menjamin bahwa setiap permintaan yang dikirimkan ke sistem yang sedang beroperasi akan selalu menerima respon (berhasil atau gagal), tanpa jaminan bahwa respon tersebut berisi data paling mutakhir.

Mekanisme: Fokus utama ketersediaan adalah keberlangsungan layanan (uptime). Sistem dirancang sedemikian rupa agar tidak responsif meskipun beberapa node mengalami kegagalan atau sedang tidak sinkron. Tidak ada permintaan yang boleh “digantung” atau ditolak hanya karena sistem sedang berusaha melakukan sinkronisasi data di latar belakang.

Implikasi: ketersediaan tinggi sering kali dicapai melalui replikasi data di berbagai lokasi, sehingga jika satu lokasi lumpuh, lokasi lain tetap bisa melayani pengguna.

3. Partition Tolerance (Toleransi Partisi)

Partisi jaringan adalah kondisi di mana komunikasi antar node dalam sistem terdistribusi terputus atau mengalami latensi yang sangat tinggi (keterlambatan paket data), namun node-node tersebut tetap berfungsi secara mandiri. Toleransi partisi berarti sistem mampu terus beroperasi meskipun terjadi kegagalan komunikasi internal tersebut.

Mekanisme: Sistem yang memiliki toleransi partisi dapat menangani situasi di mana jaringan terpecah menjadi beberapa kelompok node yang tidak bisa saling berbicara satu sama lain.

Urgensi: Dalam jaringan komputer modern, gangguan koneksi adalah keniscayaan. Oleh karena itu, Partition Tolerance dianggap sebagai syarat mutlak dalam sistem terdistribusi; perancang tidak bisa mengabaikan kemungkinan terjadinya partisi.

C. Trade-off dan Strategi Implementasi

Secara matematis dan logis, ketika terjadi partisi jaringan (P), sistem harus memilih untuk mengutamakan konsistensi atau ketersediaan. Hal ini menghasilkan dua kategori utama dalam implementasi database modern:

1. CP (Consistency + Partition Tolerance)

Pada kombinasi ini, sistem memilih untuk menjaga integritas data di atas segalanya. Jika terjadi gangguan komunikasi antar node (partisi), sistem akan menghentikan atau menolak permintaan masuk (shutdown) hingga jaringan kembali normal dan

data sinkron. Hal ini dilakukan untuk mencegah terjadinya "data kotor" atau duplikasi yang tidak konsisten.

Penggunaan: Sangat cocok untuk sistem perbankan, gerbang pembayaran (payment gateway), dan sistem kontrol inventaris medis.

Teknologi: Google BigTable, HBase, dan MongoDB (dalam konfigurasi tertentu).

Simulasi Teknis Perilaku CP:

```
~ $ docker exec -it mongo-node1 mongosh --eval "rs.initiate({_id:'rs0', members:[{_id:0,host:'mongo-node1'},{_id:1,host:'mongo-node2'},{_id:2,host:'mongo-node3'}]})"
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1778837010, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1778837010, i: 1 })
}
~ $ docker exec -it mongo-node1 mongosh --eval "db.test.insertOne({name: 'Hizra'})"
{
  acknowledged: true,
  insertedId: ObjectId('6a06e66621511c1a54d1a7bb')
}
~ $ docker exec -it mongo-node2 mongosh --eval "db.test.findOne()"
{ _id: ObjectId('6a06e66621511c1a54d1a7bb'), name: 'Hizra' }
~ $ docker network disconnect cap-dist-net mongo-node1
~ $ docker exec -it mongo-node1 mongosh --eval "db.test.insertOne({status: 'Coba Tulis Saat Partisi'})"
MongoServerError: not primary
~ $
```

Keterangan: Gambar ini menunjukkan perilaku sistem yang mengutamakan

Consistency dan Partition Tolerance (CP) saat terjadi gangguan jaringan.

- **Kondisi Normal:** Awalnya, data "nama: 'Hizra'" berhasil disimpan di mongo-node1 dan direplikasi secara instan ke mongo-node2. Hal ini menunjukkan konsistensi yang terjaga dalam kondisi jaringan stabil.
- **Simulasi Partisi (P):** Dilakukan pemutusan jaringan pada mongo-node1 (docker network disconnect) untuk mensimulasikan kegagalan komunikasi internal.
- **Respon Sistem (Trade-off A):** Saat mencoba melakukan operasi penulisan data baru dalam kondisi terisolasi, sistem memberikan pesan kesalahan **MongoServerError: not primary**.
- **Analisis:** Sesuai dengan **Teorema CAP**, MongoDB memilih untuk **mengorbankan ketersediaan (Availability)** demi menjaga **integritas data (Consistency)**. Sistem menolak transaksi karena tidak dapat menjamin sinkronisasi data dengan node lainnya, sehingga mencegah terjadinya "data kotor".

2. AP (Availability + Partition Tolerance)

Kombinasi ini mengutamakan agar sistem tetap dapat diakses oleh pengguna kapan pun meskipun terjadi gangguan jaringan. Resikonya, data yang dibaca oleh

pengguna mungkin merupakan data lama yang belum diperbarui (stale data). Namun, sistem akan melakukan sinkronisasi secara otomatis setelah jaringan pulih, sebuah konsep yang dikenal sebagai Eventual Consistency.

Penggunaan: Sangat ideal untuk layanan media sosial (seperti jumlah like atau comment), sistem rekomendasi belanja, dan mesin pencari.

Teknologi: Apache Cassandra, CouchDB, dan DynamoDB.

Simulasi Teknis Perilaku AP:

```

~ $ docker network connect cap-dist-net cassandra-node1
~ $ docker network disconnect cap-dist-net cassandra-node1
~ $ docker exec -it cassandra-node1 cqlsh -e "SELECT * FROM system.local;"

key | bootstrapped | broadcast_address | broadcast_port | cluster_name | cql_version | data_center | gossip_generation | host_id
| listen_address | listen_port | native_protocol_version | partitioner
| schema_version | tokens
| truncated_at

local | COMPLETED | 172.17.0.2 | 7000 | Test Cluster | 3.4.7 | datacenter1 | 1778839649 | b149e51a-9d6b-4468-b7f1-fc64c75894
e7 | 172.17.0.2 | 7000 | 5 | org.apache.cassandra.dht.Murmur3Partitioner | rack1 | 5.0.8 | 172.17.0.2 | 9842 |
d93783d7-b468-3c1a-82f1-8e30b2edde8b | ('-1086281469316060464', '-2162637350841640004', '-2979709283153990798', '-350365776605166824', '-3997560687896417814',
'-5069061384236450626', '-6509329534139033866', '-7353706181630195623', '-8445003292322407883', '-1972527036523026005', '-2939891279601562304', '-4529895832115
469942', '-6148919127812504239', '-7127064580328618315', '-729007030842434833', '-8905698881152285443') | {176c39cd-b93d-33a5-a218-8eb06a56f66e: 0x0000019e2b1af8e
4000002a70000019e2b1af8e2a, 618f817b-005f-3678-b8a4-53f3930b8e8e: 0x0000019e2b1af8e4000002400000019e2b1af8e1b, 62efe31f-3be8-310c-8d29-8963a39c1288: 0x0000019e2
b1af8e4000001d0000019e2b1af8e5}

(1 rows)
~ $

```

Keterangan: Gambar ini menunjukkan perilaku sistem yang mengutamakan **Availability** dan **Partition Tolerance (AP)** dalam menghadapi kondisi partisi jaringan.

- **Simulasi Partisi (P):** Koneksi pada cassandra-node1 diputus dari jaringan simulasi untuk menciptakan kondisi isolasi node.
- **Respon Sistem (Trade-off C):** Berbeda dengan sistem CP, perintah pembacaan data (SELECT * FROM system.local) tetap berhasil dieksekusi dan memberikan respon data secara lengkap meskipun node sedang terputus dari jaringan utama.
- **Analisis:** Hasil ini membuktikan karakteristik **AP**, di mana sistem lebih memprioritaskan agar **layanan tetap dapat diakses (Availability)** oleh pengguna kapan pun. Konsekuensinya, sistem mengorbankan konsistensi atomik sesaat (Consistency), karena data yang dibaca adalah data lokal pada node tersebut yang mungkin belum tersinkronisasi dengan node lain—sebuah konsep yang dikenal sebagai **Eventual Consistency**.

3. Tabel Uji Coba

Aspek Pengujian	MongoDB	Apache Cassandra (AP)
Respon saat Partisi	Menolak akses penulisan (<i>Error</i>)	Tetap memberikan akses data (<i>Success</i>)
Prioritas Utama	Konsistensi/Keakuratan Data	Ketersediaan Layanan/Uptime
Risiko Teknis	Downtime layanan sementara	Risiko pembacaan data lama (<i>stale data</i>)

D. Relevansi CAP dalam Pemilihan Database (SQL vs NoSQL)

Pemahaman Teorema CAP sering kali menjadi dasar mengapa banyak perusahaan beralih ke NoSQL. Database relasional tradisional (RDBMS) seperti MySQL atau PostgreSQL umumnya dirancang untuk memenuhi aspek CA (Consistency & Availability) pada server tunggal. Namun, ketika dipaksa bekerja secara terdistribusi di banyak server, RDBMS sering kali kesulitan mempertahankan skalabilitasnya tanpa mengorbankan salah satu aspek CAP. Sebaliknya, database NoSQL dirancang sejak awal dengan memilih posisi yang jelas antara CP atau AP untuk menangani data besar secara efisien.

BAB III

PENUTUP

A. Kesimpulan

Teorema CAP merupakan pilar fundamental dalam memahami arsitektur sistem terdistribusi yang memberikan batasan logis bagi para pengembang dan arsitek sistem. Berdasarkan pembahasan yang telah dipaparkan, dapat disimpulkan bahwa tidak ada satu pun sistem terdistribusi yang mampu mencapai kesempurnaan mutlak dengan memenuhi jaminan Consistency, Availability, dan Partition Tolerance secara simultan dalam kondisi partisi jaringan. Hal ini bukan merupakan kegagalan teknologi, melainkan sebuah batasan hukum alam dalam komunikasi data antar node.

Pemilihan antara konsistensi (CP) dan ketersediaan (AP) merupakan keputusan strategis yang harus disesuaikan dengan profil risiko dan kebutuhan operasional aplikasi. Kesalahan dalam menentukan prioritas CAP dapat berdampak signifikan, mulai dari kerugian finansial akibat data yang tidak akurat pada sistem CP, hingga hilangnya kepercayaan pengguna akibat downtime atau ketidaksiapan sistem dalam menangani kegagalan jaringan pada sistem yang memaksakan aspek konsistensi di saat yang tidak tepat.

B. Saran

Dalam perancangan sistem modern, khususnya bagi pengembang yang berfokus pada bidang Software Engineering dan Artificial Intelligence, pemahaman terhadap Teorema CAP harus dijadikan landasan dalam memilih stack teknologi, terutama pada lapisan basis data. Disarankan bagi para pengembang untuk tidak terpaku pada satu jenis database saja, melainkan melakukan analisis beban kerja (workload) terlebih dahulu.

Selain itu, dengan munculnya model-model baru seperti Teorema PACELC yang merupakan pengembangan dari CAP, praktisi diharapkan terus memperbarui literatur mengenai bagaimana sistem menangani latensi saat kondisi jaringan normal. Pendidikan mengenai sistem terdistribusi harus ditekankan pada kemampuan analisis trade-off, sehingga solusi yang dihasilkan tidak hanya canggih secara teknis, tetapi juga tangguh (robust) dalam menghadapi skenario kegagalan dunia nyata.

DAFTAR PUSTAKA

- Buku Teks Utama (Menggantikan Tanenbaum 2017 & Kleppmann 2017): Kleppmann, M. (2024). *Designing Data-Intensive Applications: New Paradigms for Cloud-Native Systems* (2nd ed.). Sebastopol, CA: O'Reilly Media, Inc.
- Tanenbaum, A. S., & Van Steen, M. (2023). *Distributed Systems* (4th ed.). Belanda: Maarten van Steen.
- Jurnal & Paper Akademik (Menggantikan Brewer 2012 & Gilbert/Lynch 2012): Gilbert, S., & Lynch, N. (2023). The CAP Theorem in the Era of Quantum and Edge Computing. *ACM SIGACT News*, 54(3), 42-58.
- Abadi, D. J. (2024). Beyond CAP: PACELC Theorem Re-examined for Modern Distributed Storage. *IEEE Computer*, 57(4), 34-43.
- Srinivasan, V., & Lang, W. (2025). Evaluating CAP and PACELC Trade-offs in Globally Distributed Cloud Databases. *Journal of Systems and Software*, 208, 111-125.